Name : Ashwin Rai Roll No. : 202403069 Class : TYCS - A

Practical No. 1 Implementing Substitution and Transposition Ciphers

Aim

To design and implement algorithms for encryption and decryption using classical substitution and transposition cipher techniques.

Objectives

1. To understand the working of substitution ciphers.
2. To understand the concept of transposition ciphers.
3. To compare substitution and transposition techniques.

Theory

1. Substitution Cipher

A substitution cipher replaces each character of the plaintext with another character according to a fixed rule.

Example: Caesar Cipher

Formula:

Encryption:

$$C = (P + K) \mod 26$$

Decryption:

$$P = (C - K) \mod 26$$

Where:

- P = Plaintext character
- C = Ciphertext character
- K = Key (Shift Value)

Example:

Plaintext: HELLO
Key: 3

Ciphertext: KHOOR

2. Transposition Cipher

A transposition cipher rearranges the positions of characters without changing the actual characters.

Example: Columnar Transposition Cipher

Plaintext: COMPUTER
Key: 4

Arrange text in rows:

C O M P

U T E R

Read column-wise:

Ciphertext: CUOTMETPR

Algorithm

A. Caesar Cipher Encryption

1. Read plaintext and key.
2. For each character:
    - Shift character by key positions.
    - Wrap around if exceeds Z.
3. Display ciphertext.

B. Caesar Cipher Decryption

1. Read ciphertext and key.
2. Shift each character backward by key positions.
3. Display plaintext.

C. Columnar Transposition Encryption

1. Read plaintext and number of columns.
2. Fill characters row-wise.
3. Read characters column-wise.
4. Display ciphertext.

D. Columnar Transposition Decryption

1. Create matrix column-wise.
2. Read row-wise.
3. Obtain original plaintext.

Program 1: Caesar Cipher

```
def encrypt(text, key):
    result = ""

    for char in text:
        if char.isalpha():
            shift = key % 26

            if char.isupper():
                result += chr((ord(char) - 65 + shift) % 26 + 65)
            else:
                result += chr((ord(char) - 97 + shift) % 26 + 97)
        else:
            result += char

    return result


def decrypt(text, key):
    return encrypt(text, -key)


plaintext = input("Enter Plain Text: ")
key = int(input("Enter Key: "))

cipher = encrypt(plaintext, key)
print("Encrypted Text:", cipher)

original = decrypt(cipher, key)
print("Decrypted Text:", original)
```

Sample Output

Enter Plain Text: HELLO
Enter Key: 3

Encrypted Text: KHOOR
Decrypted Text: HELLO

```
========== RESTART: C:/Users/CC2_50/Documents/CIS_Prac1.py ==========
Enter Plain Text: Hello
Enter Key: 3
Encrypted Text: Khoor
Decrypted Text: Hello
```

Program 2: Columnar Transposition Cipher

```
import math

def encrypt(message, key):
    cols = len(key)
    rows = math.ceil(len(message) / cols)

    matrix = []
    index = 0

    for i in range(rows):
        row = []
        for j in range(cols):
            if index < len(message):
                row.append(message[index])
            else:
                row.append('X')      # Padding
            index += 1
        matrix.append(row)

    cipher = ""
    for k in key:
        col = int(k) - 1
        for r in range(rows):
            cipher += matrix[r][col]

    return cipher
```

Name : Ashwin Rai Roll No. : 202403069 Class : TYCS - A

```
def decrypt(cipher, key):
    cols = len(key)
    rows = math.ceil(len(cipher) / cols)

    matrix = [[" for _ in range(cols)] for _ in range(rows)]

    index = 0

    for k in key:
        col = int(k) - 1
        for r in range(rows):
            matrix[r][col] = cipher[index]
            index += 1

    plain = ""
    for r in matrix:
        plain += ".join(r)

    return plain.rstrip('X')

message = input("Enter Plain Text: ").replace(" ", "").upper()
key = input("Enter Column Order (Example: 3142): ")

cipher = encrypt(message, key)
print("Encrypted Text:", cipher)

plain = decrypt(cipher, key)
print("Decrypted Text:", plain)
```

```
========== RESTART: C:/Users/CC2_50/Documents/CIS_Prac1B.py ==========
Enter Plain Text: Ashwin
Enter Column Order (Example: 3142): 3142
Encrypted Text: HXAIWXSN
Decrypted Text: ASHWIN
```